TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos

Profesores:

Ing. Jorge Arun Zambrano Cedeño. Mg.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: Silvia Patricia Carreño Cevallos

PARALELO: "A"

SEMESTRE: Quinto Semestre

ACTIVIDAD: # 1: Estudio de caso – sistema de Control de Rescate y Adopción Animal

PERIODO SEPTIEMBRE 2025 - ENERO 2026

Actividades a desarrollar en la Unidad 1

Actividad # 1

Sistema de Control de Rescate y Adopción Animal

Introducción

El rescate y la adopción de animales requieren una organización confiable de la información para registrar ingresos, identificar especies, asignar responsables y dar seguimiento a cada caso. Un proceso manual puede ocasionar duplicidad de registros, pérdida de información y dificultades para conocer el estado de los animales. Se propone un Sistema de Control de Rescate y Adopción Animal que centralice expedientes, asignaciones, tratamientos, notificaciones y solicitudes de adopción, aplicando UML, principios SOLID, patrones de diseño y Gitflow.

2. Objetivos

Objetivo general: Diseñar la arquitectura lógica de un sistema de gestión de rescate y adopción animal mediante diagramas UML, patrones de diseño y Gitflow, aplicando buenas prácticas para obtener una solución mantenible y escalable. Objetivos específicos: • Identificar actores y requerimientos funcionales y no funcionales. • Representar estructura y comportamiento con diagramas UML. • Aplicar patrones de diseño que favorezcan la creación de expedientes y las notificaciones. • Organizar el trabajo en un repositorio público de GitHub con ramas y commits trazables.

3. Análisis del problema

El refugio necesita registrar animales reportados o rescatados —perros, gatos y fauna silvestre—, almacenar fecha y ubicación, asignar voluntarios o veterinarios y actualizar el estado del caso. Los estados propuestos son Reportado, Rescatado, En tratamiento, Disponible para adopción, Adoptado y Cerrado. El sistema deberá conservar un historial de cambios y permitir consultar casos por especie, estado o responsable.

3.1 Actores

• Ciudadano/Reportante: comunica un posible rescate y entrega los datos disponibles.

• Voluntario: consulta los casos asignados y registra actividades de rescate.

• Veterinario: registra observaciones y tratamientos de los casos asignados.

• Administrador del refugio: valida expedientes, asigna responsables, administra estados y supervisa adopciones.

• Adoptante: consulta animales disponibles y presenta solicitudes de adopción.

3.2 Requerimientos funcionales

RF01 Registrar un caso de rescate. RF02 Clasificar al animal por especie. RF03 Guardar fecha, ubicación, descripción y datos relevantes. RF04 Asignar voluntario o veterinario. RF05 Actualizar el estado del caso. RF06 Notificar a responsables autorizados sobre nuevos casos y asignaciones. RF07 Registrar y evaluar solicitudes de adopción. RF08 Buscar expedientes por estado, especie o responsable. RF09 Conservar historial de modificaciones. RF10 Controlar permisos según el rol del usuario.

3.3 Requerimientos no funcionales

RNF01 Escalabilidad: admitir más casos, usuarios y especies. RNF02 Mantenibilidad: separar responsabilidades en componentes. RNF03 Seguridad: proteger datos mediante autenticación y autorización. RNF04 Trazabilidad: conservar el historial de cambios y versiones. RNF05 Usabilidad: ofrecer formularios claros. RNF06 Integridad: validar datos y evitar registros inconsistentes.

4. Arquitectura lógica propuesta

La arquitectura se divide en cuatro capas. Presentación (vistas y formularios); Control/Aplicación (controladores y coordinación de casos); Dominio (expedientes, usuarios, asignaciones y adopciones); e Infraestructura (repositorios, base de datos y notificaciones). MVC separa la interfaz de la lógica; las interfaces reducen el acoplamiento entre el dominio y los servicios externos. El diseño es una propuesta lógica, no una implementación funcional completa.

4.1 Principios SOLID

SRP: cada clase tiene una responsabilidad principal. OCP: se pueden incorporar nuevas especies mediante nuevas clases que respeten la abstracción ExpedienteAnimal. LSP: las clases concretas deben respetar el contrato de la clase base. ISP: las interfaces deben ser pequeñas y específicas. DIP: los servicios de alto nivel dependen de interfaces, porejemplo RepositorioCasos, no de una base de datos concreta.

5. Diagrama de casos de uso

La Figura 1 representa las interacciones principales de los actores con la plataforma: reportar rescates, consultar casos, registrar expedientes, asignar responsables, actualizar estados, registrar tratamientos, notificar novedades y gestionar adopciones. Los permisos finales se implementarán según el rol de cada usuario.

DIAGRAMA UML DE CASOS DE USO

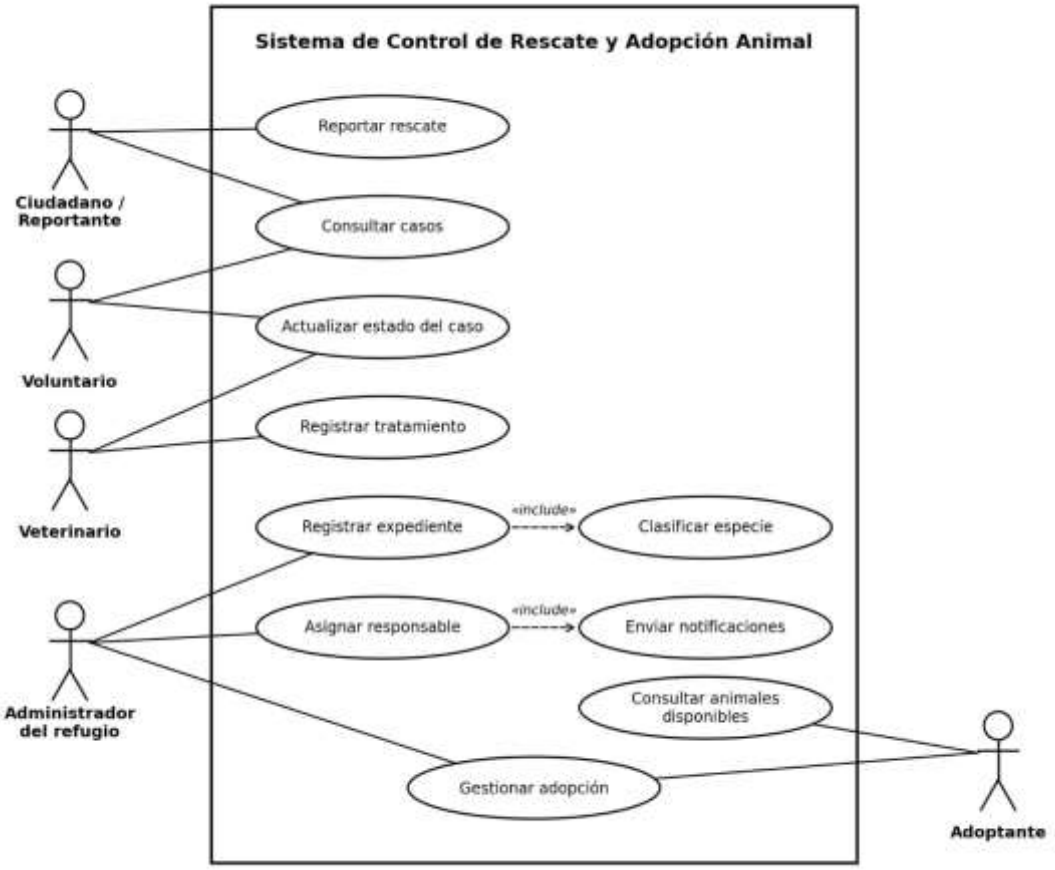


Figura 1. Diagrama UML de casos de uso

6. Diagrama de clases

La Figura 2 presenta las clases principales y sus relaciones. Usuario es una abstracción de la que derivan Ciudadano, Voluntario, Veterinario y Administrador. ExpedienteAnimal representa el caso general y las clases concretas amplían los datos específicos de perro, gato y fauna silvestre. Asignacion registra al responsable; SolicitudAdopcion documenta la petición; ServicioNotificacion distribuye eventos; y FabricaExpedientes abstrae la creación por especie.

DIAGRAMA DE CLASES UML

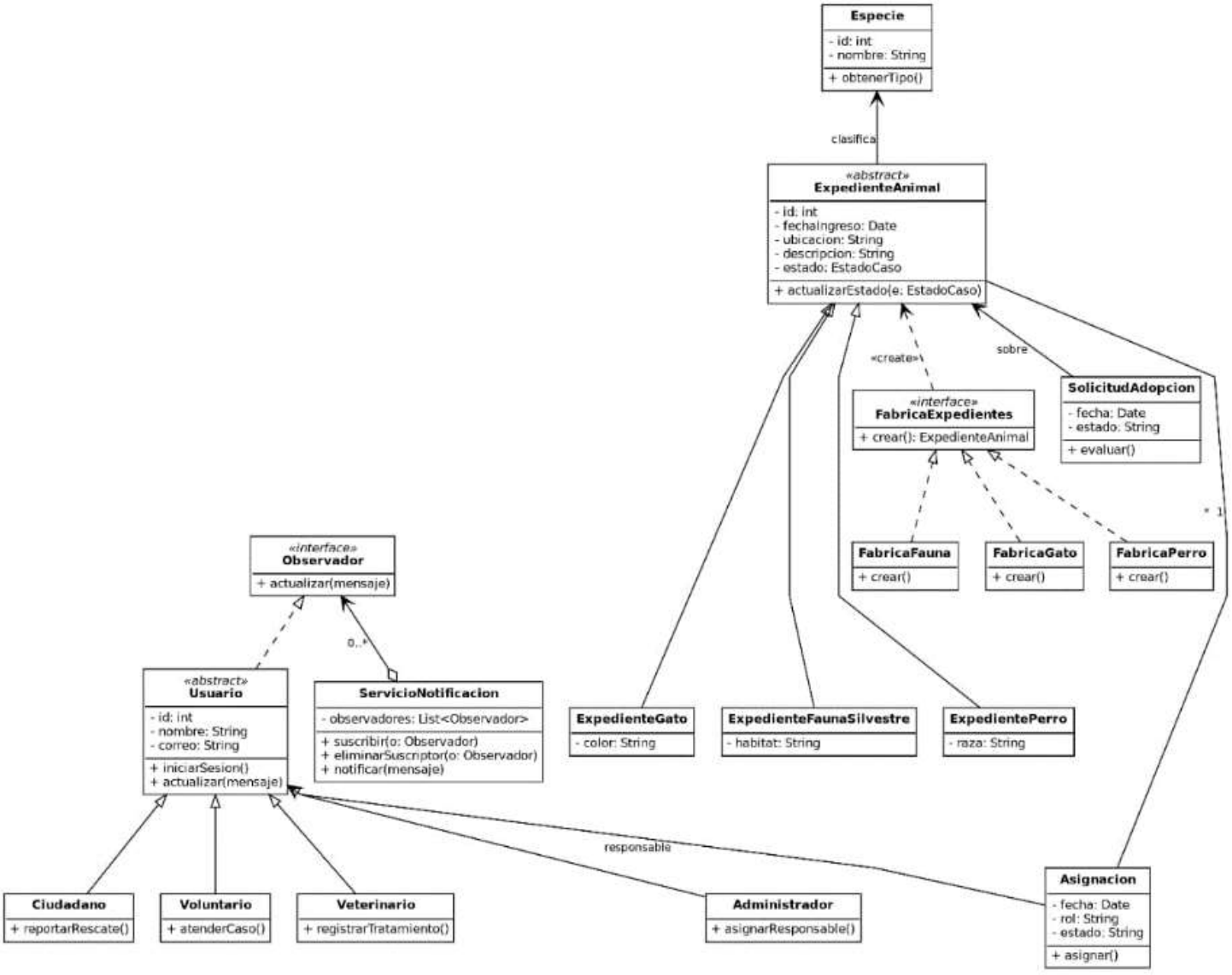


Figura 2. Diagrama de clases UML. Elaboración propia.

7. Diagrama de secuencia

La Figura 3 muestra el registro de un rescate. El ciudadano introduce los datos en la vista; el controlador envía la solicitud al servicio de casos; la fábrica crea el expediente apropiado; el repositorio guarda la información; y el servicio de notificaciones comunica el evento a los suscriptores autorizados. En una implementación real, deben manejarse errores de validación y fallos de persistencia antes de confirmar el registro.

DIAGRAMA DE SECUENCIA UML: REGISTRO DE RESCATE

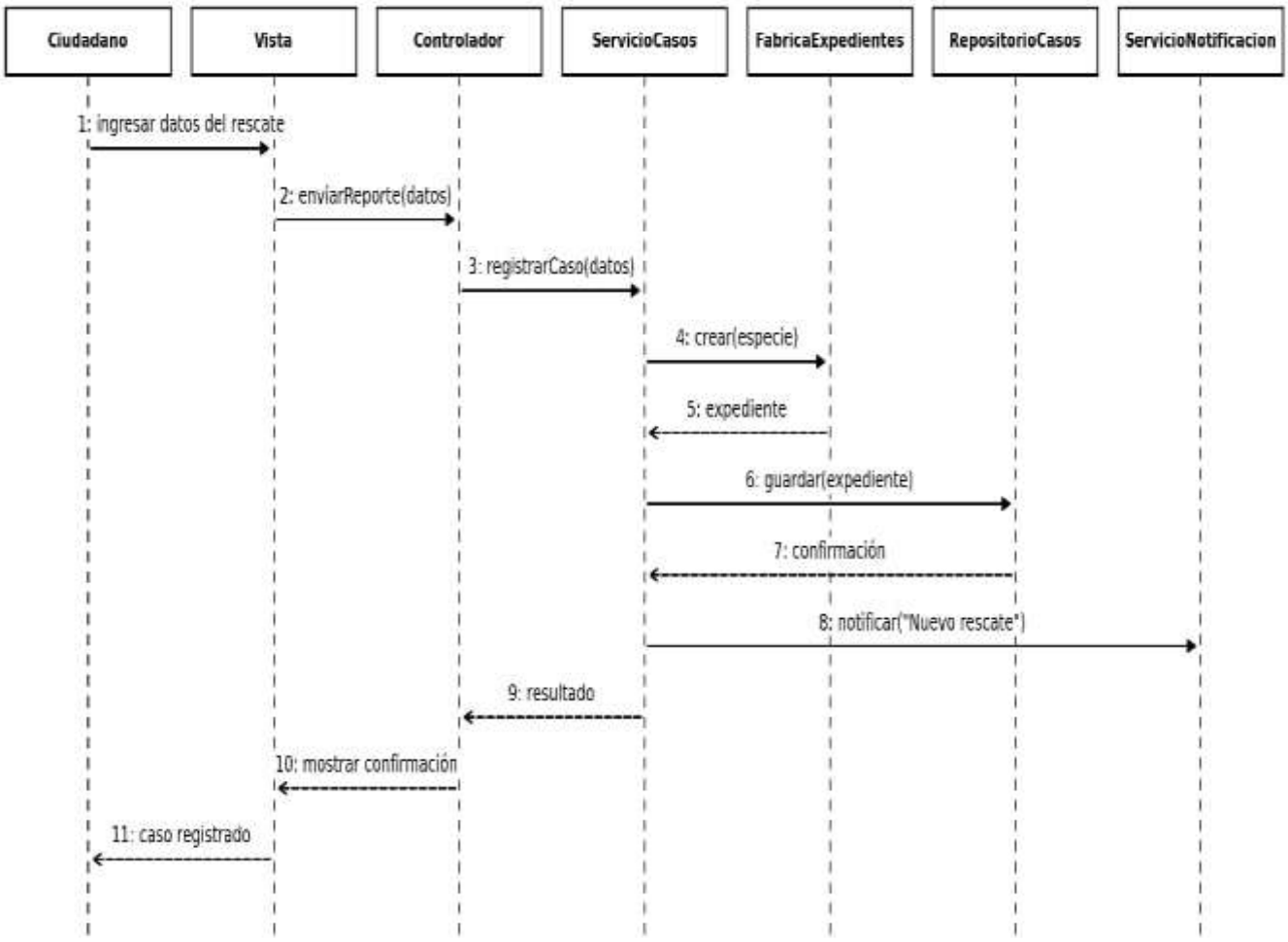


Figura 3. Diagrama UML de secuencia: registro de un rescate

8. Patrones de diseño

Singleton: se propone para una configuración o coordinador compartido que realmente requiera una única instancia. No se recomienda convertir todas las clases ni la conexión de base de datos en singleton; para conexiones es preferible un pool y para pruebas, dependencias sustituibles. Observer: el servicio de notificaciones publica eventos cuando se registra un rescate o cambia una asignación. Los observadores suscritos reciben la notificación sin acoplar el servicio de casos a cada canal de comunicación. Factory Method: una abstracción FabricaExpedientes permite crear ExpedientePerro, ExpedienteGato o ExpedienteFaunaSilvestre según la especie seleccionada. Añadir otra especie requerirá una nueva implementación respetando el contrato común. MVC: Modelo contiene entidades y reglas; Vista presenta formularios y resultados; Controlador recibe acciones y coordina servicios. La separación permite modificar la interfaz sin reescribir el dominio.

9. GitHub y Gitflow

Se creará un repositorio público llamado sistema-rescate-adopcion-animal. La rama master conservará versiones estables; develop integrará avances; las ramas feature/analisis-requerimientos, feature/diagramas-uml, feature/patrones-diseno y feature/arquitectura contendrán trabajos específicos. Cada feature se crea desde develop y, tras revisión, se integra de nuevo a develop. Una versión revisada se integra a master. Commits sugeridos: docs: agregar analisis del problema; docs: definir requerimientos; docs: agregar diagramas UML; docs: documentar patrones; docs: explicar flujo Gitflow. Estos son ejemplos: se deben ejecutar los commits reales y adjuntar capturas del historial**. Enlace real del repositorio**
https://github.com/SilviaCa1988/Deber_DSI

.

10. Seguridad y mantenimiento

Se validarán los datos antes de guardar, se limitará el acceso por roles y se registrarán las operaciones relevantes. Los datos personales de reportantes y adoptantes se protegerán y usarán solo para fines autorizados. Se recomienda probar transiciones de estado, asignaciones, creación por especie y notificaciones; mantener la documentación actualizada; y no almacenar contraseñas en texto plano.

11. Conclusiones

El sistema propuesto organiza el ciclo de rescate y adopción mediante expedientes trazables y responsabilidades definidas. UML permite comunicar la estructura y el comportamiento esperado; SOLID favorece una arquitectura menos acoplada; y Singleton, Observer, Factory Method y MVC resuelven necesidades específicas cuando se aplican con criterio. GitHub y Gitflow proporcionan un historial verificable de los avances. La propuesta deberá complementarse con evidencias reales del repositorio y una revisión de los diagramas antes de la entrega.

12. Referencias

Atlassian. (s. f.). Gitflow workflow. https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.

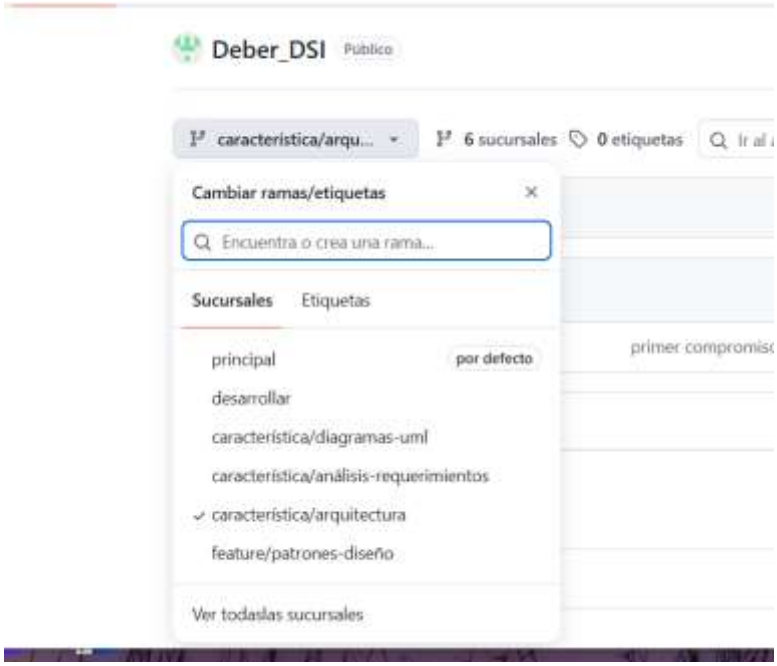
GitHub Docs. (s. f.). GitHub documentation. https://docs.github.com/

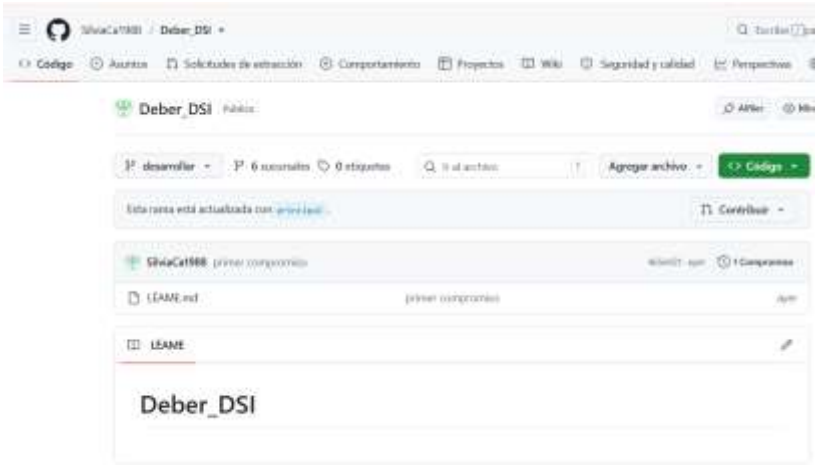
Martin, R. C. (2017). Clean architecture: A craftsman's guide to software structure and design. Prentice Hall.

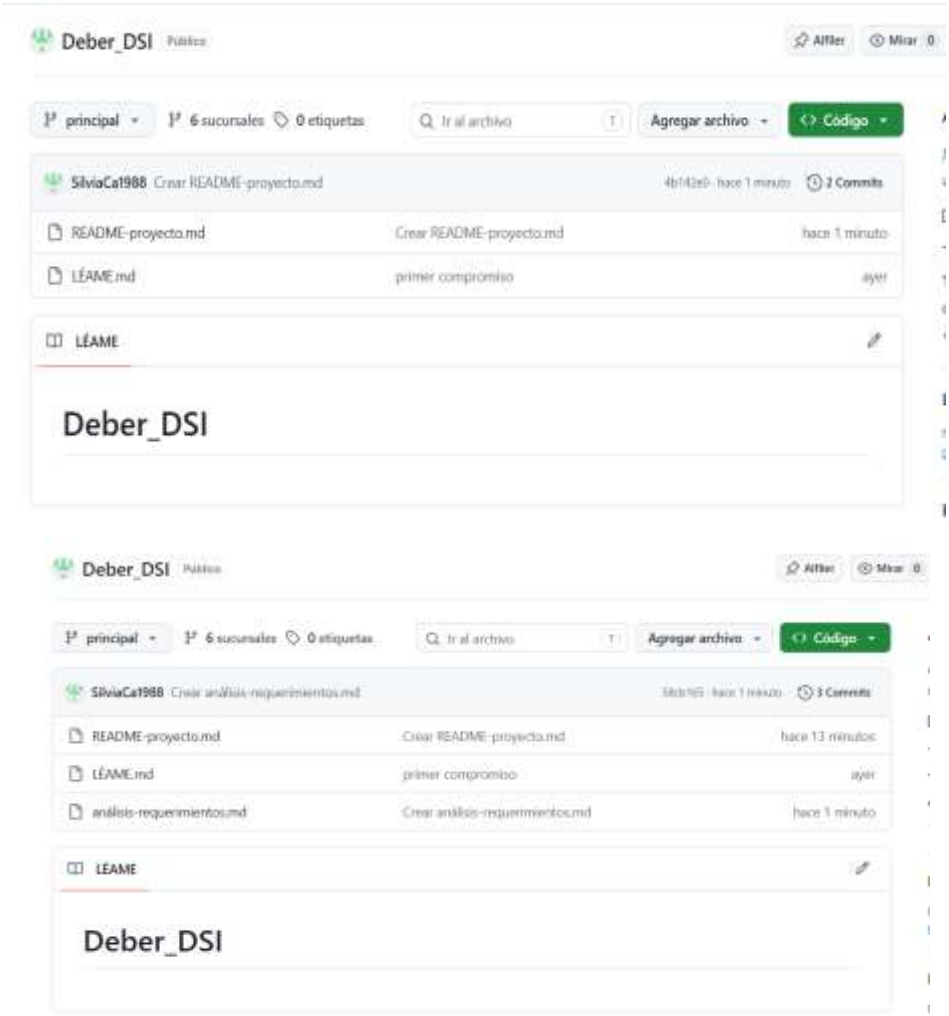
Object Management Group. (2017). OMG Unified Modeling Language (OMG UML),Version 2.5.1. https://www.omg.org/spec/UML/2.5.1

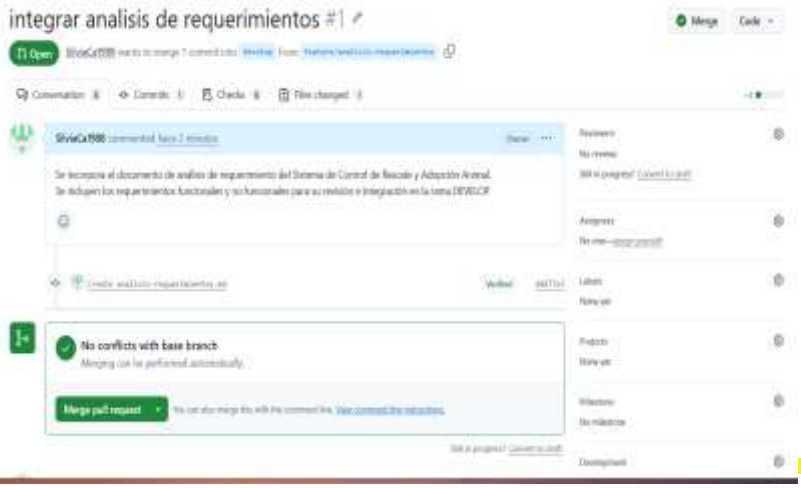
Evidencias que se añadirán: repositorio creado, ramas master/develop, ramas feature, historial de commits y una integración de feature a develop.

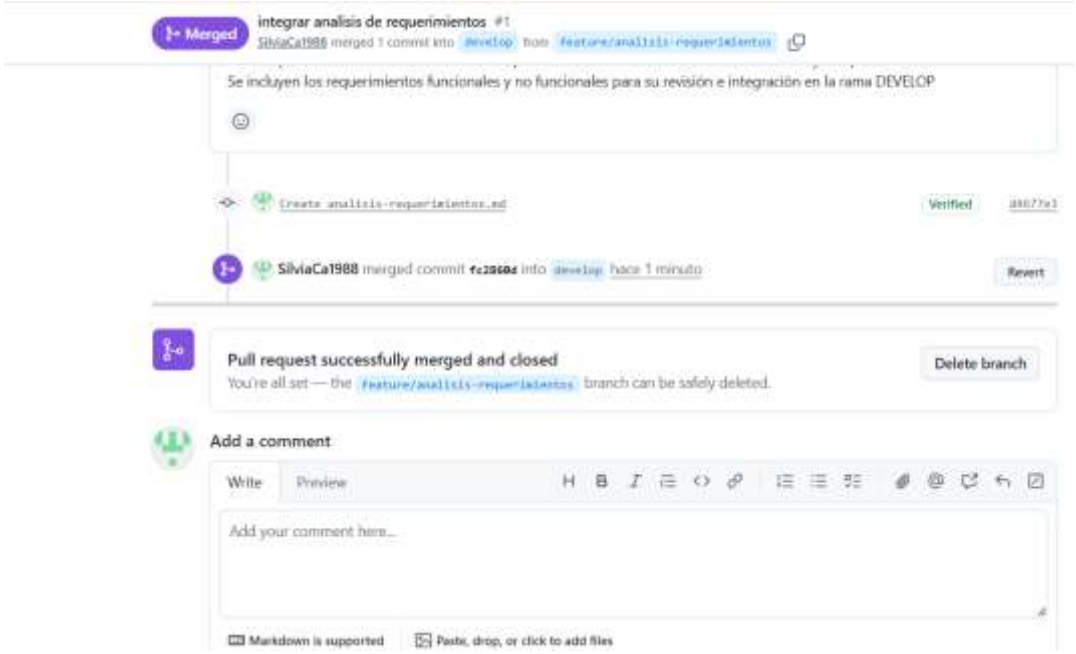
Evidencias del trabajo en Github






Deber_DSI
principal
6 sucursales
0 etiquetas
Ir al archivo
Agregar archivo
Código
SilviaCa1988 Crear README-proyecto.md
2 Commits
README-proyecto.md
Crear README-proyecto.md
hace 1 minuto
LÉAME.md
primer compromiso
ayer
LÉAME
Deber_DSI
Deber_DSI
principal
6 sucursales
0 etiquetas
Ir al archivo
Agregar archivo
Código
SilviaCa1988 Crear análisis-requerimientos.md
3 Commits
README-proyecto.md
Crear README-proyecto.md
hace 13 minutos
LÉAME.md
primer compromiso
ayer
análisis-requerimientos.md
Crear análisis-requerimientos.md
hace 1 minuto
LÉAME
Deber_DSI


integrar analisis de requerimientos #1
Open
No conflicts with base branch
Merge pull request


Merged
integrar analisis de requerimientos #1
SilviaCa1988 merged 1 commit into develop from feature/analisis-requerimientos
Se incluyen los requerimientos funcionales y no funcionales para su revisión e integración en la rama DEVELOP
Create analisis-requerimientos.md
Verified
SilviaCa1988 merged commit fc2868d into develop hace 1 minuto
Revert
Pull request successfully merged and closed
You're all set — the feature/analisis-requerimientos branch can be safely deleted.
Delete branch
Add a comment
Write
Preview
Add your comment here...
Markdown is supported
Paste, drop, or click to add files